Lctr 4: Data Representation

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Expression of negative integers in two's complement (TC):
  - Encoding in both binary and decimal.
  - Decoding in several ways; decimal-based approach.
- C's data types of different bit widths.
- Endianness of multi-byte data.
- Memory map

Demo project:

- Printing out the TC encoding of negative numbers.
- Printing out the contents of various data types.

Additional reading for review of fundamentals of C

- C's scalar variables, arrays, structs, and unions.

4.1 Introduction

4.2 Demo project

4.3 Signed integers

4.3.1 Three ways to encode signed integers

To express **signed** integers, we can split the entire range of 0 to 15 into two halves: 0 to 7 and 8 to 15, which will be used to express **signed** numbers in the following way:

- 0 to 7 are used to represent the same nonnegative (or positive for short) values as before;
- 8 to 15 are used to represent negative values.

Three approaches to **encode** signed numbers: **Sign-and-magnitude** (SnM), **One's complement** (OC), and **Two's complement** (TC).

Binary	SnM	OC	TC	unsigned Decimal	Binary	SnM	OC	TC
0b0000	+0	+0	0	8	0b1000	-0	-7	-8
0b0001	1	1	1	9	0b1001	-1	-6	-7
0b0010	2	2	2	10	0b1010	-2	-5	-6
0b0011	3	3	3	11	0b1011	-3	-4	-5
0b0100	4	4	4	12	0b1100	-4	-3	-4
0b0101	5	5	5	13	0b1101	-5	-2	-3
0b0110	6	6	6	14	0b1110	-6	-1	-2
0b0111	7	7	7	15	0b1111	-7	-0	-1

We can see that for all the three approaches, **the MSB is 0 for positive numbers and 1 for negative ones**. As such, the MSB is called the **sign bit**.

OC for -2
① -2 = 2 ⇒ 0b0010
② 0b1101

4.3.2 Comparison of the above three expressions

TC for -2 is OC for -2 +1

0b1101
0b1110

4.4 Two's complement (TC) expression

There are two similar and confusing terms regarding TC in the literatures:

- Converting x to TC.
- Calculating or taking TC of x.

The true meanings of the above terms are as follows.

- Converting x to TC means to find x's TC representation in binary form. This can be called **encoding**, which will be used when confusion can happen.
- Calculating or taking TC of x means to compute the negative value of x. This is used to convert a subtraction to addition, as $a - b = a + (-b)$. This can be referred to as **negating the number using the TC expression**.

4.4.1 Binary-based hand calculation for TC encoding

We use an example to review the binary-based approach.

Example 1. Encode -3 in TC using a 5-bit system using the *standard* binary approach.

Solution:

Using the binary approach, we have:

- The magnitude of -3 is 3, which has a binary form as $s_{\text{P}} =$ `0b00011`.
- Inverting every bit of the above binary sequence, we have the OC of -3 as $s_{\text{OC}} =$ `0b11100`.
- Adding 1 to the OC, we have TC as $s_{\text{TC}} =$ `0b11101`.

-:-	0b1101	-5	-2	-3

If we compare -3's 5-bit TC encoding, which is $s_{\text{TC},5} =$ `0b11101`, with its 4-bit TC encoding, which is $s_{\text{TC},4} =$ `0b1101` from a previous table, we see that the only difference is that it has one more 1 at the MSB side for the 5-bit encoding. If we use a 6-bit system, the TC encoding of -3 will be $s_{\text{TC},6} =$ `0b111101`. On the other hand, the TC encoding of 3 will be `0b0011`, `0b00011`, and `0b000011`, for 4-, 5-, and 6-bit expressions. The point is: **when we extend the number of bits, we repeat the MSB**.

4.4.2 Decimal-based hand calculation for TC encoding

We can also do TC encoding using an **unsigned decimal**-based approach, which can be simpler than the standard binary approach in most cases. To see how this approach works, let's reconsider the steps for 5-bit TC encoding of -3.

00011
~11100

11111
00011
11100

- **Encoding -3 in OC**: Inverting each bit of its magnitude, 3, can be seen as `0b1_1111- 0b0_0011 = 0b1_1100` or `31- 3= 28`. As if this unsigned
- **Adding 1 to OC**: It is equivalent to `0b1_1111- 0b0_0011+ 1= 0b10_0000- 0b0_0011 = 0b1_1101` or `32- 3= 29`. Note that $32 = 2^5$ and we have used decimal to binary conversion to obtain the above binary sequence. As if this is unsigned.

$-3 = 29 - 32$

This can be extended to the general case of N-bit systems. Let α be a positive integer, and we want to encode $-\alpha$ in TC. Then, the encoding in *unsigned decimal* is represented as $\bar{\alpha}_N = 2^N - \alpha$. This is illustrated in the table below for $\alpha = 1, 2, ..., 8$ and $N = 4, 5$.

-α	4-bit TC	$\bar{\alpha}_4$	5-bit TC	$\bar{\alpha}_5$
-1	0b1111	15	0b11111	31
-2	0b1110	14	0b11110	30
-3	0b1101	13	0b11101	29
-4	0b1100	12	0b11100	28
-5	0b1011	11	0b11011	27
-6	0b1010	10	0b11010	26
-7	0b1001	9	0b11001	25
-8	0b1000	8	0b11000	24

This ties back to the concept of *encoding*: a binary sequence of `0b1110` can represent 14 if we see the number as unsigned; we can also see it as -2 if we consider the number as signed.

[0, 15] [16, 31] unsigned

[-16, -1] [0, 15]

-32

+32 = 25

-15 ⇒ -15+32=17 ⇒ 0b1_0001

Example 2: Perform the following operations using a 5-bit system **using the decimal-based approach**. Give the solution in **both decimal and binary** forms as needed. (You can use the demo code of this lecture to verify the results.)

1. Encode the following numbers in TC: `5`, `-6`, `-14`, and `12`.
2. Negate and encode the following numbers in TC: `5`, `-7`, `-12`, `3`.

Solution:

① ⓐ 5 ⇒ Dec: 5; Bin: 0b0_0101
ⓑ -6 ⇒ Dec: -6+32=26; Bin: 0b1_1010
ⓒ -14 ⇒ Dec: -14+32=18; Bin: 0b1_0010
ⓓ 12 ⇒ Dec: 12; Bin: 0b0_1100
② ⓐ 5 ⇒ Dec: -5 ⇒ -5+32=27; Bin: 0b1_1011
ⓑ -7 ⇒ Dec: 7; Bin: 0b0_0111.
ⓒ -12 ⇒ Dec: 12; Bin: 0b0_1100
ⓓ 3 ⇒ Dec: -3 ⇒ -3+32=29; Bin: 0b1_1101

-:-

4.4.3 TC decoding

Sometimes, we are given a data in the computer memory encoded in TC and want to decode it—getting the value of the signed integer that it **encodes**. This can be done in many approaches, as illustrated below with the decoding of `0b11101` of a 5-bit system.

Approach 4. **Direct decimal calculation**. We will calculate $\bar{\alpha}$ first by seeing the number as unsigned and then minus 2^N. This is the same as the first part of Approach 3.

When seeing `0b11101` as unsigned, its value is $= 2^4 + 2^3 + 2^2 + 2^0 = 16 + 8 + 4 + 1 = 29$. Then, its signed value is 29 - 32 = -3.

Approach 4 can save us a lot of computation time for many cases.

Example 3: Assume we have the following 5-bit binary sequences that are TC encodings of signed integers: `0b0_0101`, `0b1_1010`, `0b1_0010`, and `0b0_1100`. Find the values they represent.

$2^5 = 32$

Solution:

- `0b0_0101`. The unsigned value is 5. MSB = 0, so the number is 5-0, which is 5.
- `0b1_1010`. The unsigned value is 26. MSB = 1, so the number is 26-32, which is -6.
- `0b1_0010`. The unsigned value is 18. MSB = 1, so the number is 18-32, which is -14.
- `0b0_1100`. The unsigned value is 12. MSB = 0, so the number is 12-0, which is 12.

> -:-

4.4.4 Advantages of using two's complement expression

- There is only a single 0 in the number system, which facilitates the zero checking operations. We will see this type operations later when we discuss status flags and condition code prefixes.
- TC simplifies hardware implementations for the MCU:
  - The hardware implementation for TC-based subtraction is the same as that for TC-based addition by taking the TC of the subtrahend: Minuend - Subtrahend = Minuend + TC(Subtrahend)
  - The hardware implementation for addition, subtraction, and multiplication for signed integers are identical to those for unsigned integers.

4.5 Equivalent definitions of C variables

Now, let's see the different data types and their ranges:

- Unsigned char, $N = 8$, RoV: 0 to 255 (0 to $2^8 - 1$)
  - `char k;`—preferred if we want to define a string.
  - `unsigned char k;`
  - `uint8_t k;`—preferred if we want to specify the number of bits (NoB).
- Signed char, $N = 8$, RoV: -128 to +127 (-2^7 to $2^7 - 1$)
  - `signed char k;`
  - `int8_t k;`—preferred if we want to specify the NoB.
- Unsigned short, $N = 16$, RoV: 0 to 65535 (0 to $2^{16} - 1$)
  - `unsigned short k;`
  - `uint16_t k;`—preferred if we want to specify the NoB.
- Signed short, $N = 16$, RoV: -32768 to +32767 (-2^{15} to $2^{15} - 1$)
  - `short k;`
  - `signed short k;`
  - `int16_t k;`—preferred if we want to specify the NoB.
- Unsigned int, $N = 32$, RoV: 0 to 4294967295 (0 to $2^{32} - 1$)
  - `unsigned int k;`
  - `unsigned long k;` lu
  - `uint32_t k;`—preferred if we want to specify the NoB.
- Signed int, $N = 32$, RoV: -2147483648 to +2147483647 (-2^{31} to $2^{31} - 1$)
  - `int k;`—preferred if we want a general purpose variable as it saves typing.
  - `signed int k;`
  - `signed long k;`
  - `int32_t k;`—preferred if we want to specify the NoB.

4.6 Storing different types of variables in memory

4.6.1 Memory view and data types

The data is saved in the memory in the unit of bytes, each having 8 bits. There are two aspects of the memory: the address, which is 32 bits in width, and the content, which is 8-bit wide, as shown in Figure 1.

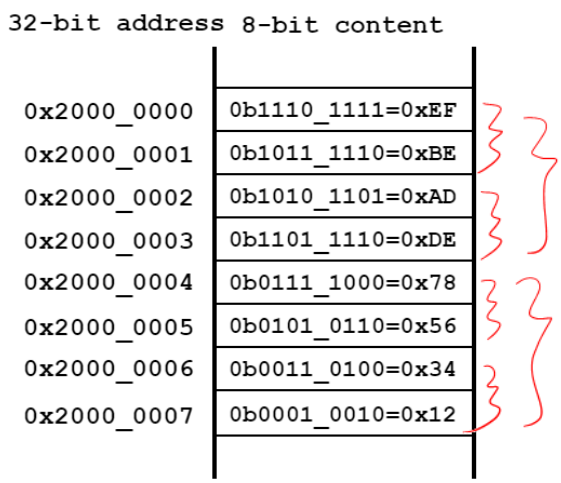

Figure 1: Memory view of addresses and contents.

4.6.1.1 8-bit numbers

Each address in the figure has a corresponding 8-bit value. For example, the number saved in memory address `0x2000_0000` is `0b1110_1111= 0xEF = 239`.

In C, these bytes are defined as `uint8_t` or `int8_t`, unsinged or signed byte. Seeing them as unsigned or signed is determined by the intention of the programmers who use the data.

4.6.1.2 Aligned 16-bit numbers

We can combine two 8-bit bytes into a `uint16_t` or `int16_t` number, which is also called an unsigned or signed **halfword**. Note that **the address of a halfword is the lower address of the two bytes**. For efficient access of data, the addresses of halfwords should be **aligned** to the boundaries of multiples of 2, and hence we can only have the following legitimate addresses for halfwords:

- `0x2000_0000`
- `0x2000_0002`
- `0x2000_0004`
- `0x2000_0006`

4.6.1.3 Aligned 32-bit numbers

Further, we can combine two halfwords into a `uint32_t` or `int32_t` number, which is also called a **word**. Note that **the address of a word is the lowest address of the four bytes**. For efficient access of data, the addresses of words should be aligned to the boundaries of multiples of 4, and we can only have the following legitimate addresses for words:

- `0x2000_0000`
- `0x2000_0004`

4.6.1.4 Non-aligned numbers

4.6.1.5 Separating address from data

Note that we have to make a clear distinction between the address and the content of the memory.

- The memory address is like the room number of a classroom in a classroom building, which is like the memory space.
- The BYTE saved in a memory location in the memory space is like a class conducting in a classroom in the classroom building.
- The value of the BYTE can change in a similar way as the class can change with time according to the schedule.

4.6.2 Endianness of numbers

The little- and big-endian formats are defined below:

- Little Endian: The most significant byte is stored at the highest address of the variable.
- Big Endian: The most significant byte is stored at the lowest address of the variable.

The above definitions can be represented graphically in Figure 2.

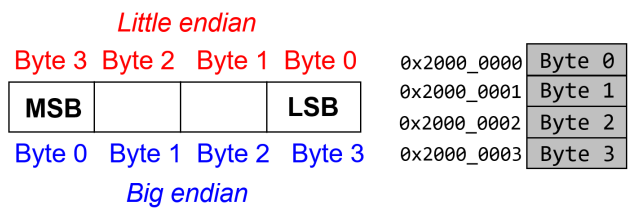

Figure 2: Little endian vs big endian.

There are two important points to note here:

- Regardless of the endianness, the address of a word is always defined as the lowest address of all bytes it occupies.
- ARM uses little-endian by default. However, it can change to big-endian by configuration. In the sequel, except in examples and exercises, we always use little-endian so that we will always be on the same page.

32 bits

Example 4: Consider the memory view of Figure 1. A. What is the value of the word saved at 0x2000_0000 in hexadecimal if we use little-endian? B. What is the value of the halfword saved at 0x2000_0006 in hexadecimal if we use big-endian?

Solution:

A. 0x DEAD_BEEF.

B. 0x 3412.

:-:

0x2000_0000	0b1110_1111=0xEF
0x2000_0001	0b1011_1110=0xBE
0x2000_0002	0b1010_1101=0xAD
0x2000_0003	0b1101_1110=0xDE
0x2000_0004	0b0111_1000=0x78
0x2000_0005	0b0101_0110=0x56
0x2000_0006	0b0011_0100=0x34
0x2000_0007	0b0001_0010=0x12

1 2 3 Increase by 1, the number is increased by 1

100 more significant

4.7 Memory maps

Figure 1 shows the standard 1-byte 1-column memory map. Here, we see several variations of the memory map.

4.7.1 1-byte 4-column memory map

Using the numbers of Figure 1, we have the following 1-byte 4-column memory map:

Address	Byte 0	Byte 1	Byte 2	Byte 3
0x2000_0000:	0xEF	0xBE	0xAD	0xDE
0x2000_0004:	0x78	0x56	0x34	0x12

The idea can be extended to any number of columns, such as 2 or 8.

0x2000_0000	0b1110_1111=0xEF
0x2000_0001	0b1011_1110=0xBE
0x2000_0002	0b1010_1101=0xAD
0x2000_0003	0b1101_1110=0xDE
0x2000_0004	0b0111_1000=0x78
0x2000_0005	0b0101_0110=0x56
0x2000_0006	0b0011_0100=0x34
0x2000_0007	0b0001_0010=0x12

4.7.2 4-byte 1-column memory map

Address	4-Byte Word
0x2000_0000:	0xDEAD_BEEF
0x2000_0004:	0x1234_5678

4.7.3 2-byte 2-column memory map

Address	Halfword 0	Halfword 1
0x2000_0000:	0xBEEF	0xDEAD
0x2000_0004:	0x5678	0x1234

4.8 Additional reading for reviewing fundamentals of C

4.8.1 Format of decimal literals

Note the following notations:

- `432U` for `uint32_t` type (where `U` stands for `unsigned long`)
- `432L` for `int32_t` type (where `L` stands for 32-bit `signed long`)

4.8.2 Scalar variables, arrays, structs, and unions

4.8.2.1 Arrays

An **array** is a set of *homogeneous* data, stored in contiguous memory locations, beginning at a named address, which is the name of the array.

```
char    str08[] = "deadbeef";
int8_t  arr08[8] = {0, 1, 2, 3, 4, 5, 6, 7};
int16_t arr16[] = {0, 1, 2, 3, 4, 5, 6, 7};
int32_t arr32[8] = {0, 1, 2, 3, 4, 5, 6, 7};
```

4.8.2.2 Structs

While an array is a collection of *homogeneous* data, a **struct** is a collection of *heterogeneous* data.

```
typedef struct {
    int8_t  nums8[2];
    int16_t num16;
    int32_t num32;
} mp_test_struct_t;
```

4.8.2.3 Unions

The definition of unions is very much like that for struct. Yet, all the elements in the union share the same memory.

```
typedef union {
    uint8_t  my_arr08[8];
    uint32_t my_arr32[2];
    mp_test_struct_t my_struct;
} my_test_union_t;
```